

Code Troubleshooting - GenAI Usage Report

1. Code Troubleshooting & Logic Guidance

Merging Excel & CSV Data on Mismatched ID Types

Question: "When joining `sensors.xlsx` and `calibrations.csv` on `sensor_id`, the merged pandas DataFrame came back empty. What went wrong?"

Solution: Excel files often store numeric IDs as integers (e.g., `101`), while CSV files may store them as strings (e.g., `'101'`). When data types don't match, merging returns zero results. Fix this by converting the `sensor_id` column to the same data type in both DataFrames before using `pd.merge()`.

Date & Data Type Mismatches During Filtering

Question: "My filtering logic wasn't returning overdue sensors even though some were outdated. Why wasn't the comparison working?"

Solution: Check the data type of your `days_since_calibration` column. If it's stored as text instead of numbers, comparisons like `'90' > '100'` will give wrong results. Use `pd.to_numeric()` to convert the column to numeric type before applying filters.

Resolving Dynamic File System Paths

Question: "My script kept throwing a `FileNotFoundError` when reading `config.yml` because VS Code was running from a different folder. How do I fix this?"

Solution: Don't use hardcoded relative paths like `"config.yml"`. Instead, use Python's `pathlib` module to find the directory containing your script with `Path(__file__).parent`, then join it with your filename. This makes your script work regardless of where it's run from.

Handling Empty Configuration Files

Question: "What happens if `config.yml` is empty, and how do I prevent crashes when looking up settings?"

Solution: PyYAML's `safe_load()` returns `None` for empty files instead of a dictionary. Add a check in your configuration loading function: if the result is `None`, return an empty dictionary `{}` instead. This prevents crashes when trying to access configuration keys.

Code Structure & Type Hints

Question: "How do I structure my code to follow PEP 8 and work with static type checkers like Pylance?"

Solution: Break your code into focused functions with clear responsibilities. Add type hints and docstrings to each function. When converting DataFrames to dictionaries with `to_dict(orient="records")`, use `typing.cast` to tell the type checker what format to expect.

2. Summary of GenAI Usage (For Submission / Canvas Log)

Overview: Used GenAI to debug logic issues in `check_sensors.py`, focusing on DataFrame data type mismatches, file path resolution with `pathlib`, and safe YAML parsing for empty files.

Key Learnings: How to merge different data sources (Excel and CSV), ensure consistent data types across DataFrames, safely handle empty configuration files, and apply proper type hints following Python best practices.